📘 DOCUMENTACIÓN TÉCNICA — MÓDULO MARKETPLACE

Proyecto: DesignForge AI

🧠 1. VISIÓN GENERAL

Se implementó un módulo de **Marketplace funcional end-to-end**, conectando:

Frontend (Next.js)
↓
Backend (FastAPI + SQLAlchemy)
↓
Base de datos (Supabase PostgreSQL)
↓
Storage (Supabase Buckets)

El sistema ya permite:

- Visualizar productos reales
- Manejar inventario dinámico
- Cargar imágenes desde storage
- Escalar a funcionalidades avanzadas (carrito, pedidos, IA, etc.)

🏗️ 2. ARQUITECTURA FINAL

🔹 Frontend

- Next.js (App Router)
- TypeScript
- Tailwind CSS
- Arquitectura desacoplada

Frontend → ProductService → Backend API → DB + Storage

🔹 Backend

- FastAPI
- SQLAlchemy
- Supabase (Auth + Storage únicamente)

Controller → Service → DB (SQLAlchemy)
→ Storage (Supabase)

🔹 Base de datos (Supabase)

- PostgreSQL
- Modelo relacional normalizado
- Integridad referencial estricta

🔹 Storage (Supabase)

- Bucket público: `product-catalog`
- Gestión de archivos mediante `file_assets`

🧩 3. MODELO DE DATOS IMPLEMENTADO

🔹 Tablas usadas

✔ `companies`

- Empresa dueña de productos
- Se creó:

id = 1
name = DesignForge

✔ `product_types`

1 → bordado
2 → neon-flex

3 → acrilico

👉 Deben coincidir EXACTAMENTE con frontend

✔ `products`

Campos clave:

- `company_id`
- `product_type_id`
- `name`
- `description`
- `base_price`
- `is_active`

✔ `inventory`

- Relación 1:1 con producto
- Define disponibilidad

quantity > 0 → inStock = true

✔ `file_assets`

Tabla crítica para manejo de archivos

Campos clave:

- `bucket_name`
- `storage_path`
- `file_type`
- `product_id`

⚠️ Regla crítica

Solo UNA FK debe estar presente:
product_id OR company_id OR order_item_id OR transaction_id

🖼️ 4. STORAGE — DISEÑO Y USO

📁 Bucket

product-catalog (público)

📂 Estructura

{company_id}/{product_id}/{filename}

Ejemplo real:

product-catalog/1/4/main.webp

📌 Tipos de archivo

- `product_main` → imagen principal

⚠️ Regla clave

NO hardcodear URLs
SIEMPRE usar storage_path

🧪 5. DATOS DE PRUEBA CREADOS

Productos

ID	Nombre	Tipo	Precio
4	Camiseta Bordada Personalizada	bordado	45000
5	Letrero Neon Flex LED	neon-flex	120000
6	Placa Acrílica Decorativa	acrilico	80000

Inventory

Product ID	Stock
4	10
5	0
6	5

Imágenes

- Producto 4 → ✔ tiene imagen
- Producto 5 → ❌ no tiene
- Producto 6 → ❌ no tiene

⚙️ 6. BACKEND IMPLEMENTADO

📄 Estructura

```
controllers/product_controller.py
services/product_service.py
schemas/product_schema.py
models/
```

📄 Schema

```
ProductResponse:
- id
- title
- description
- imageUrl
- price
- rating
- reviews
- inStock
- productType
```

📄 Service (lógica)

Query hace JOIN de:

- products
- product_types
- inventory
- reviews
- file_assets

🧠 Transformaciones clave

✔ Imagen

image_url = f"{SUPABASE_URL}/product-catalog/{storage_path}"

✔ Stock

inStock = quantity > 0

✔ Rating

avg + count

📄 Endpoint

GET /products

Respuesta:

```
[
 {
  "id": 4,
  "title": "...",
  "imageUrl": "...",
  "inStock": true
 }
]
```

🎨 7. FRONTEND INTEGRADO

🔹 Cambios realizados

❌ Antes

mockProducts.ts

✅ Ahora

fetch(`${API_URL}/products`)

📄 ProductService

getProducts(): Promise<Product[]>

📄 ProductCard

Problema solucionado:

src="" ❌

Solución:

{product.imageUrl ? <Image ... /> : <Fallback />}

🔗 8. FLUJO COMPLETO

1. Usuario entra a /marketplace
2. useProducts llama a ProductService
3. ProductService hace fetch a backend
4. Backend ejecuta query SQL
5. Une:
   - products
   - inventory
   - file_assets
6. Construye response
7. Frontend renderiza:
   - imagen
   - stock
   - precio

⚠️ 9. DECISIONES ARQUITECTÓNICAS CLAVE

✔ Backend como intermediario

Frontend → Backend → Supabase

Motivos:

- Seguridad
- Escalabilidad
- Lógica de negocio
- Integración futura con IA

✔ SQLAlchemy sobre Supabase directo

Motivos:

- Queries complejas
- Control total
- Escalabilidad

✔ file_assets desacoplado

Permite:

- múltiples imágenes
- versionado
- IA futura
- reutilización

🚀 10. ESTADO ACTUAL DEL SISTEMA

✅ COMPLETADO

- DB estructurada
- Productos reales
- Inventory funcional
- Storage conectado
- Backend operativo
- Endpoint `/products`
- Frontend consumiendo API

- UI funcional

❗ PENDIENTE

- Imágenes productos 5 y 6
- Reviews reales
- Filtros backend
- Paginación
- Carrito

🔥 11. GUÍA PARA CONTINUAR

Próximos pasos recomendados

1. Subir imágenes faltantes

1/5/main.webp
1/6/main.webp

---

2. Filtros backend

/products?type=bordado

---

3. Paginación

/products?page=1&limit=10

---

4. Reviews

- Insertar en DB
- Reflejar en frontend

---

5. Carrito

- Nueva tabla
- Nueva lógica backend

🧠 12. RESUMEN FINAL

Este módulo ya cumple con:

✔ Arquitectura desacoplada
✔ Backend robusto
✔ Base de datos normalizada
✔ Storage correctamente integrado
✔ Frontend conectado a datos reales

👉 Esto NO es un prototipo
👉 Es una base real de marketplace lista para escalar

📌 IMPORTANTE PARA EL EQUIPO

Para continuar desarrollo:

1. **NO romper `ProductService`**
2. **NO hardcodear URLs**
3. **SIEMPRE usar backend como fuente de verdad**
4. **Respetar estructura de storage**
5. **Mantener tipado consistente frontend-backend**